Autonomous Tracking in Cluttered Environments

Bruno Pinto, *Instituto Superior Técnico, Universidade de Lisboa, Portugal*

***Abstract*—This thesis presents a trajectory planning framework for autonomous aerial tracking of a moving target in cluttered environments, with a focus on maintaining target visibility while ensuring safe and dynamically feasible quadrotor motion. The work addresses the coupled challenges of perception-aware planning, obstacle avoidance, and real-time trajectory optimization, which are key requirements for applications such as aerial cinematography, inspection, and search and rescue. The proposed method adopts a two-stage architecture for motion planning consisting of a front-end and a back-end. The front-end generates a reference trajectory for tracking the target by modeling the quadrotor–target interaction as a leader-follower problem. The back-end utilizes a Model Predictive Control (MPC) formulation to compute smooth, feasible, and safe trajectories that satisfy the nonlinear quadrotor dynamics and actuator limits. The MPC incorporates multiple soft objectives: maintaining a constant distance to the target, minimizing field-of-view (FOV) occlusion, enforces collision avoidance via a repulsive potential cost, and aligns the quadrotor's FOV with the target. To evaluate the performance of the framework, a wide range of simulated experiments were conducted. These include scenarios designed to verify tracking behavior, obstacle avoidance experiments to evaluate the collision-avoidance cost, and dense, cluttered environments to test the planner's ability to maintain visibility while navigating through constrained areas. Additional experiments analyze the sensitivity of the system to cost weights, prediction horizon, and noisy target measurements. Results demonstrate that the proposed approach consistently achieves collision-free tracking, maintains target visibility by reducing occlusion, even in cluttered environments. The MPC solver achieves computation times suitable for online execution, validating the feasibility of the method for real-time autonomous tracking.**

***Index Terms*—Trajectory Planning, Target Tracking, Model Predictive Control, Occlusion Minimization, Optimization**

I. Introduction

In recent years, with the advancement of vision and navigation technology, quadrotors have been widely used in commercial applications: wind turbines inspection, surveillance, search and rescue, 3D surface reconstruction, exploration, cinematography and videography. Common to all those applications is the ability of the quadrotor to navigate avoiding obstacles and point the onboard camera to points of interest.

The main goal of this thesis is to develop a trajectory planning framework to track a moving target (i.e. a person, another drone) while avoiding obstacles in a cluttered environment. There are four main technical challenges of autonomous aerial tracking:

1) The trajectory of the drone should be collision free from obstacles and a safe distance to the target should be kept at all times.
2) The drone should keep the target in its front camera's field-of-view (FOV) and avoid occlusions by surrounding obstacles.

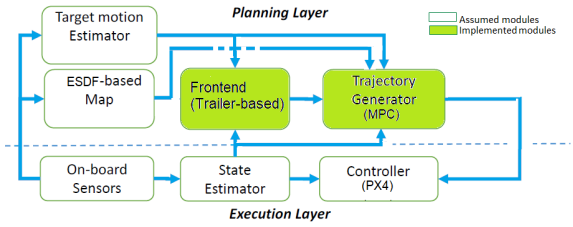


Fig. 1. Proposed System Architecture.

3) Smoothness: The trajectory should be smooth to enable quadrotor tracking and minimize motion blur.
4) The whole framework should run fast enough to allow for online execution.

For aerial tracking, a setting with the following assumptions, is considered:

1) The quadrotor has an omni-directional (360º) camera onboard to detect, initially unknown, obstacles.
2) The target is expected to have smooth motion with bounded velocity and acceleration.
3) The target's position and velocity are estimated online.
4) The target pose, quadrotor state and map of the environment can be estimated online.
5) The terrain is assumed to be flat, allowing for 2-D planning, with constant height above the ground.
6) Only one target is tracked a given time.

The quadrotor must track the moving target from a constant distance while avoiding obstacles. A two step approach is proposed for the tracking: front-end and back-end. The front-end is responsible for generating the reference trajectory without considering obstacles and vision; The back-end tracks the reference trajectory while avoiding obstacles and maximizes visibility by solving a trajectory optimization problem.

The trajectory planner must generate a smooth, dynamically feasible, and collision-free trajectory that follow a visually-tracked moving target. The trajectory planner must run online,in a receding horizon fashion. As visual observations of the target and the environment are obtained, the target future trajectory can be estimated and used to design a trajectory to follow the target. The proposed system architecture is shown in Fig. 1.

Colored modules corresponds to the modules implemented in this work. The remaining modules are assumed to be available.The target motion estimator is the ground truth, the state estimator is the simulator ground truth. In practice, the quadrotor has a flight controller onboard that receives the control inputs from the trajectory generator and outputs motor commands. The goal of this thesis is the design of the front-end and the trajectory generator.

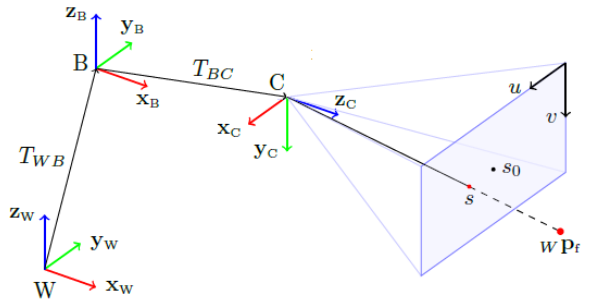


Fig. 2. Illustration of the world frame W, the body frame B and the camera frame C. [1]

II. Background

In this section, we present the background necessary to understand the methods developed in this thesis. We begin by introducing the quadrotor model and the reference frames used to describe its motion. Next, we provide a brief overview of motion planning, outlining the typical division into front-end path generation and back-end trajectory planning. Finally, we describe the trajectory optimization problem at a high level, highlighting how feasibility, safety, and perception objectives are incorporated into the formulation.

A. Quadrotor Model

In this thesis, we make use of a world frame $\mathcal{W}$ with orthonormal basis $X_{\mathcal{W}}$, $Y_{\mathcal{W}}$, $Z_{\mathcal{W}}$. The quadrotor body frame $\mathcal{B}$, has orthonormal basis $X_{\mathcal{B}}$, $Y_{\mathcal{B}}$, $Z_{\mathcal{B}}$. The quadrotor is equipped with a non-gimbaled camera, whose reference frame C has orthonormal basis $X_{\mathcal{C}}$, $Y_{\mathcal{C}}$, $Z_{\mathcal{C}}$. In this thesis, we assume that the camera frame is aligned with the body frame. Fig. 2 illustrates the different reference frames. A quaternion is used to represent frame orientations.

The robot state is $\boldsymbol{x} = (\boldsymbol{p}, \boldsymbol{v}, \boldsymbol{q})^T$ where $\boldsymbol{p} \in R^3$ is the position of the robot's center of mass (COM) with respect to $\mathcal{W}$, $\boldsymbol{q} \in \mathrm{R}^4$ is the orientation of the body frame expressed in $\mathcal{W}$, $\boldsymbol{v}$ is the COM linear velocity expressed in $\mathcal{W}$.

The input is $u = (\boldsymbol{c}, \boldsymbol{\Omega_B})$ where $\boldsymbol{c} = (0, 0, c_z)^T$ is the mass-normalized thrust vector with $c_z = \frac{(f1+f2+f3+f4)}{m}$, f_i is the thrust produced by the i-th motor, and m is the mass of the vehicle. $\boldsymbol{\Omega_B} = (w_x, w_y, w_z)^T$ is the angular velocity expressed in $\mathcal{B}$. According to [2] the quadrotor dynamics can then be modelled as:

$$\begin{aligned} \dot{p} &= v \\ \dot{v} &= g + q \otimes \boldsymbol{c} \\ \dot{q} &= \frac{1}{2} S(\boldsymbol{\Omega_B}) \cdot q \end{aligned} \tag{1}$$

where $g = (0, 0, -9.81)^T$ ms^{-2} is the gravity acceleration vector in $\mathcal{W}$, and S is the skew-symmetric operator. The symbol $\otimes$ is the Hamilton operator for quaternions multiplication.

More complex quadrotor models exist that incorporate aerodynamic drag and propeller speeds or actuation in torque, however a simple model is sufficient for trajectory generation and re-planning purposes. Such is the case for this thesis. The model parameters used in this thesis are summarized in table I.

TABLE I
System Parameters

Parameter	Symbol	Value	Unit
Mass	m	0.36	kg
Distance from CoM to rotor	l	0.23	m
Moment of inertia (x-axis)	I_{xx}	0.0348	kg·m^2
Moment of inertia (y-axis)	I_{yy}	0.0459	kg·m^2
Moment of inertia (z-axis)	I_{zz}	0.0977	kg·m^2

B. Motion Planning

This thesis focus on the classical approach of dividing the planning problem as a front-end and a back-end.

The front-end finds a collision-free path by either sampling-based ([3], [4]) or search-based methods [5], some methods are purely geometric (i.e. only considers position), others take into account the dynamics.

The back-end step enhances the path found by the front-end by employing trajectory optimization techniques, to generate a safe, continuous, and dynamically feasible trajectory by solving an optimization problem, which may include control costs, collision costs, and constraints such as kino-dynamic feasibility and actuator saturation.

1) Frontend planning: We consider two main approaches to finding the reference trajectory, one is a leader-follower approach, the other is based on path planning by using algorithms like sampling-based ([3], [4]) or search-based methods [5]. This methods takes into account the obstacles to find a collision-free trajectory. To adapt these methods for target tracking, additional terms can be added in the cost function.

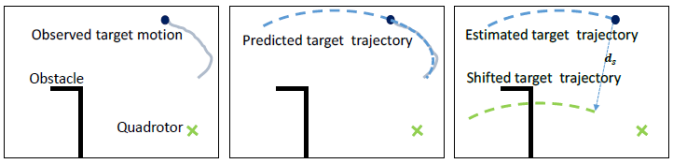
The leader-follower approach does not consider obstacles. Given the leader's position the follower tracks the leader at a distance d with an angle θ. This behavior is represented by:

$$\mathbf{p}_{\mathcal{F}}(t) = \mathbf{p}_{\mathcal{L}}(t) - \mathcal{R}(t) d(t) \tag{2}$$

d(t) is the Euclidean distance between the leader and the follower, expressed in the body frame of the follower and θ(t) is the relative planar angle between the target and quadrotor in the inertial frame, which in 2-D defines the rotation matrix $\mathcal{R}(t)$. Both values can be time-varying. The goal is to select the appropriate functions for both.

a) Constant Distance:
One trivial solution is to set d(t) as a constant and choose θ such that the follower's trajectory is a mere copy of the leader's trajectory, with a constant displacement, i.e. $\theta_F = \theta_L$. This is equivalent to maintaining a relative position $\mathbf{d}_s \in \mathrm{R}^3$ to the moving target. This is the approach used in [6], illustrated in Fig. 3.

This approach has several issues: It is sensitive to obstacles, sensitive to noisy target's position measurements and in this work we are interested in tracking a moving target right-behind, not a mere offset.



(a) Observe the target motion using on-board sensors. (b) Use polynomial fitting to estimate and predict the target trajectory (Sect. III). (c) Shift the predicted target trajectory by $\mathbf{d}_s$ (Sect. V-A).

Fig. 3. Graphical illustration of the tracking offset. [6]

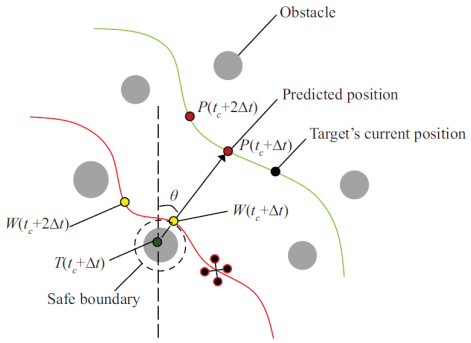


Fig. 4. Point with the constant distance is unfeasible, the feasible point lies on the bearing vector but closer to the leader. [6]

b) Time-varying distance:
The work in [7] sets d(t) as a constant in the first stage and choose θ as mere offset of the leader. To account for collisions, d(t) is chosen so as to minimize the distance between the follower and the leader and lies on the bearing vector, P_L - P_F, shown in Fig. 5

c) Angle-varying distance:
The same work provides another approach. It sets d(t) as a constant and selects the angle θ, considering occlusion by obstacles by solving the following optimization problem:

$$\min l_1 + \lambda_1 \mathrm{e}^{-(l_2+d_m)} + \lambda_2 \left\| l_3 - d_t \right\| \tag{3}$$

where SFC is the safe flight corridor (no obstacles).

The first term penalizes the Euclidean distance l_1 from current position $W\left(t_c\right)$ at t_c to next desired waypoint $W\left(t_c + \Delta t\right)$ at $t_c + \Delta t$ with the predicting period Δt. The second term maximizes the closest distance l_2 from bearing vector to the obstacles. The third term penalizes the error between the distance from the desired waypoint and the constant tracking distance. The optimization problem is illustrated in Fig. 5.

Although the resulting trajectory minimizes occlusion, avoids obstacles and is insensitive to noise, due to the limited FOV this leads to local minimums and jagged reference trajectories. Another big limitation is that the optimization problem scales linearly with the number of obstacles.

2) Trajectory Optimization Back-end: Let $\mathbf{x}$ and $\mathbf{u}$ be the state and input vectors of the quadrotor, respectively. The dynamics is described by a set of first order differential equations $\dot{\mathbf{x}} = \boldsymbol{f}(\mathbf{x}, \mathbf{u})$. Let $\mathbf{z} = f_p(\mathbf{x}, \mathbf{u}, \sigma)$ be the state vector of the perception system, that depends on the quadrotor state,

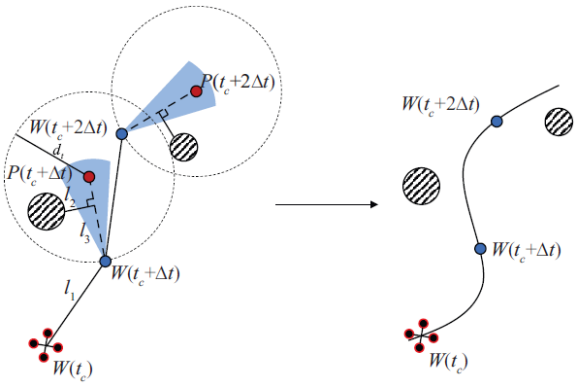


Fig. 5. Illustration of the optimization problem to minimize occlusion. [6]

control, and σ a vector of parameters characterizing the camera system (e.g., camera calibration matrix or its field of view cone). A cost function, $\mathcal{L}_a(\mathbf{x}, \mathbf{u})$, can defined given control objectives, for example tracking a reference trajectory. The cost function $\mathcal{L}_p(\mathbf{z})$ is defined for the perception objectives. The cost function $\mathcal{L}_c(\mathbf{x}, \mathbf{z})$ represents the collision costs. Notice how the collision cost may depend on the perception. Given the estimated target position in time, the optimizer's goal is to generate a trajectory that ensures the target's visibility, guaranteeing safety, collision-free and dynamic feasibility. Can then formulate the coupling of perception and action as an optimization problem:

$$\begin{aligned} \min_{\mathbf{u}} \int_{t_0}^{t_f} & \mathcal{L}_a(\mathbf{x}, \mathbf{u}) + \mathcal{L}_p(\mathbf{z}) + \mathcal{L}_c(\mathbf{x}) dt \\ \text{subject to} \quad & \dot{\mathbf{x}} = \mathrm{f}(\mathrm{t}, \mathbf{x}, \mathbf{u}) \\ & \mathrm{r}(\mathbf{x}, \mathbf{u}, \mathbf{z}) = 0 \\ & \mathrm{h}(\mathbf{x}, \mathbf{u}, \mathbf{z}) \leq 0, \end{aligned} \tag{4}$$

where $\dot{\mathrm{x}} = \boldsymbol{f}(\mathrm{t}, \mathbf{x}, \mathbf{u})$ represents the continuous dynamic, $\mathrm{r}(\mathbf{x}, \mathbf{u}, \mathbf{z})$ and $\mathrm{h}(\mathbf{x}, \mathbf{u}, \mathbf{z})$ represent equality and inequality constraints that the solution should satisfy for perception, action and collision. For example, bounds on velocity and acceleration, control input.

The optimization problem can be convex or non-convex depending on the type of cost functions and constraints. There are many implementation decisions: design the cost functions, how to define the trajectory, which map representation to use, how to initialize the optimization, how tackle the occlusion problem. Depending on the type of optimization problems, there are several issues to solve: low-quality initial guess, solving in real-time (if the solution even exists), handling sensor noise and address local minima.

III. Target Tracking Methodology and Implementation

A. Control Architecture

The proposed control architecture is illustrated in Fig. 6.

The planner (front-end) receives information about the target's position and linear velocity; current quadrotor's state and outputs the reference position, velocity and the reference yaw.

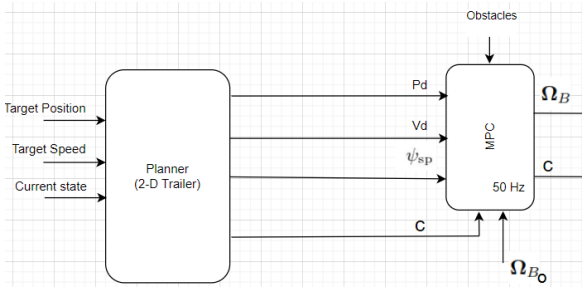


Fig. 6. MPC framework overview.

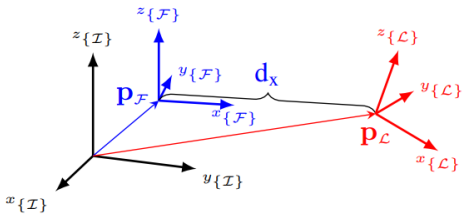


Fig. 7. Leader-Follower coordinate frames [8].

The back-end planner is implemented as a nonlinear model predictive controller. It uses a predictive model of the quadrotor and takes into account the reference trajectory from the front-end, the current measured or estimated vehicle state, and the positions of surrounding obstacles. At each control step, the controller solves an optimization problem to minimize trajectory tracking errors while satisfying dynamic constraints, actuator limitations, and collision avoidance requirements. The computed control outputs are normalized thrust and body rate commands, which are directly compatible with a flight controller which finally converts the commands into motor speed commands.

B. Target's motion prediction

This thesis focuses exclusively on the motion planning component of the control architecture, therefore, no online motion prediction or estimation module for the target is included. Instead, the complete target trajectory is assumed to be fully known a priori. This assumption removes uncertainties related to target behavior, allowing the planning framework to be developed and evaluated under deterministic conditions.

The entire target trajectory is available offline and is represented using a differentiable cubic spline. During operation, the planner accesses the next N target waypoints in a receding-horizon fashion, enabling evaluation of tracking performance without reliance on estimation modules.

C. Front-end: 2-D Trailer

Lets consider the task of target tracking without considering obstacles, for now. This task can be formulated as the Leader-Follower approach to formation control. The objective is for a follower vehicle (the quadrotor) to remain at a, potentially, time-varying relative position and orientation in a given reference frame, with respect to the Leader vehicle (the target). The problem is characterized by the relative position vector and the reference frame where it is defined, illustrated in Fig. 7. P_F is the position of the quadrotor and P_L is the position of the target.

A two part solution to the Leader-Follower formation problem in here proposed. First, a desired trajectory is computed for an ideal Follower vehicle, hereafter called virtual Follower. This reference trajectory is then used as a reference to a trajectory tracking controller that drives the real Follower vehicle to the desired trajectory.

The proposed approach is to fix d(t) by treating the follower as a 2-D trailer attached to the leader via a longitudinal link of length d [9], as shown in Fig. 7 and define the time evolution of the rotation of the longitudinal link.

The position, p_F is defined by:

$$\mathbf{p}_{\mathcal{F}}(t) = \mathbf{p}_{\mathcal{L}}(t) - {}^{\mathcal{I}}_{\mathcal{F}}\mathcal{R}(t)\mathbf{d} \tag{5}$$

$\mathbf{p}_F$ is the position of the follower and $\mathbf{p}_L$ is the position of leader, both expressed in the inertial frame, $\mathcal{R}(t)$ is the rotation matrix between follower frame to the inertial frame, parametrized by the angle ψ_F, and $\mathbf{d} = [\mathrm{d}_x \ 0]^T$ is a constant distance vector along the longitudinal axis. The time derivative of (5) is given by

$$\dot{\mathbf{p}}_{\mathcal{F}}(t) = \dot{\mathbf{p}}_{\mathcal{L}}(t) - {}^{\mathcal{I}}_{\mathcal{F}}\mathcal{R}(t)S(\boldsymbol{\omega}_{\mathcal{F}}(t))d, \tag{6}$$

where S($\boldsymbol{\omega}_{\mathcal{F}}$) is the skew-symmetric matrix, $\boldsymbol{\omega}_{\mathcal{F}}$ is the angular velocity of the follower.

Assuming that the follower moves only longitudinally, consider $\mathbf{u}_{\mathcal{F}} = [\mathrm{u}_F \ 0]^T$ is the follower's body-frame velocity, expressed in F. Solving (6) for $\boldsymbol{u}_F$ and $\boldsymbol{\omega}_{\mathcal{F}}$ yields

$$\mathbf{u}_{\mathcal{F}}(t) = \begin{bmatrix} 1 & 0 \end{bmatrix} \ {}^{\mathcal{I}}_{\mathcal{F}}\mathcal{R}(t)\dot{\mathbf{p}}_{\mathcal{L}}(t) \tag{7}$$

$$\boldsymbol{\omega}_{\mathcal{F}}(t) = \frac{1}{d_x}\begin{bmatrix} 0 & 1 \end{bmatrix} \ {}^{\mathcal{F}}_{\mathcal{I}}\mathcal{R}(t)\dot{\mathbf{p}}_{\mathcal{L}}(t) \tag{8}$$

Equations (7), (8) completely defines the follower's trajectory, which is not a mere offset of the leader's trajectory. The follower's trajectory is the result of propagating equations (7), (8) for N points in a receding horizon fashion.

In the context of the planning strategy developed in this thesis, the leader (target) trajectory $(P_L(t), \dot{P}_L(t))$ is not predicted online but is instead obtained directly from a cubic spline that fits the target's waypoints. At each planning instant t, the spline provides the planner with N future target states in a receding horizon fashion:

$$\left\{ P_L(t + i\Delta t),\ \dot{P}_L(t + i\Delta t) \right\}_{i=0}^{N-1}. \tag{9}$$

D. Back-end: Model Predictive Control

Model predictive control (MPC) [10] is an optimization-based control strategy. The MPC uses a mathematical model to predict the future behavior of the system over a finite time horizon and compute an optimal control input that satisfies a set of constraints while minimizing a cost function. The control input is calculated by solving at each time step a finite horizon open-loop optimal control problem, then the

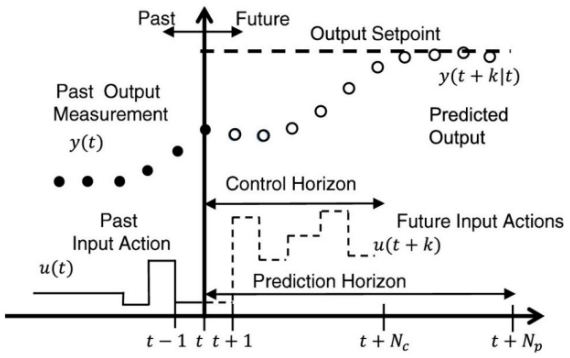


Fig. 8. MPC receding horizon concept. Image credits: [10]

first action from the calculated control input is applied to the system until the next time step. The dynamics evolve for one step and the procedure repeats.

In more detail, the MPC framework works as follows: (1) obtain an estimate of the current state with a state estimator, (2) optimize a trajectory starting from the current state, (3) execute the first action from the optimized trajectory, (4) let the dynamics evolve for one step and repeat. This recipe is known as model-predictive control, illustrated in Fig. 8.

For this thesis the following optimization problem is formulated:

$$\begin{aligned} &\min_{\mathbf{x},\mathbf{u}} \overline{\mathbf{x}}_N^\top \mathcal{Q}_{x,N} \overline{\mathbf{x}}_N + \sum_{i=1}^{N-1} \left(\overline{\mathbf{x}}_i^\top \mathcal{Q}_{x,i} \overline{\mathbf{x}}_i + \overline{\mathbf{u}}_i^\top \mathcal{R}_i \overline{\mathbf{u}}_i \right) + J_{sc} \\ &\text{s.t. } \dot{\mathbf{x}} = \mathrm{f}(\mathrm{t}, \mathbf{x}, \mathbf{u}) \text{ (dynamic model)} \\ &\quad c_{\min} \leq \mathbf{c} \leq c_{\max} \text{ (normalized thrust bounds)} \\ &\quad -\Omega_{\max} \leq \mathbf{\Omega_B} \leq \Omega_{\max} \text{ (body rate command bounds)} \end{aligned} \tag{10}$$

where the values $\overline{x}$ and $\overline{u}$ refer to the difference with respect to the reference of each value. The reference for the states and inputs are given by a target pose or a pre-computed trajectory (that neglects the perception objectives). $J_{sc} = \lambda_{tck} J_{tck} + \lambda_{col} J_{col} + \lambda_{occ} J_{occ} + \lambda_{fov} J_{fov}$. The costs J_{tck}, J_{col} and J_{occ} are the tracking, collision, occlusion costs and fov alignment, respectively.

1) Tracking Cost: The tracking cost is defined as:

$$J_{tck} = \sum_{k=0}^{N-1} (d_k - d_{dth})^2 \tag{11}$$

$$d_k = \|\mathbf{P}_k - \mathbf{T}_k\|, k \in \{0, 1, \ldots, N-1\}, \tag{12}$$

where P and T are the quadrotor and target 3D position, respectively. d_{dth} is the distance the quadrotor should keep from the target, defined as 2 meters, which corresponds to half of the FOV distance.

2) Collision Cost: The collision cost is formulated as the repulsive force of the obstacles acting on each waypoint:

$$J_{coll} = \sum_{k=0}^{N-1} F_c \left(d \left(\mathbf{P}_k \right) \right), \tag{13}$$

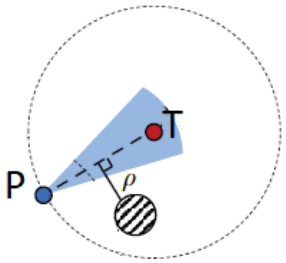


Fig. 9. Illustration of the optimization problem to minimize occlusion. [6]

where $\mathrm{d}(\mathrm{P}_k)$ is the distance between P_k and the closest obstacle. Fc is a differentiable potential cost function:

$$F_c \left(d \left(\mathbf{P}_k \right) \right) = \begin{cases} \left(d \left(\mathbf{P}_k \right) - d_{thr} \right)^2 & d \left(\mathbf{P}_k \right) \leq d_{thr} \\ 0 & d \left(\mathbf{P}_k \right) > d_{thr} \end{cases} \tag{14}$$

where d_{thr} is specifying the threshold of obstacle clearance.

The collision cost acts as the repulsive force that pushes the quadrotor trajectory away from obstacles. In this thesis, the target is also considered an obstacle to penalize heavilly if the quadrotor flies too close to the target.

3) Occlusion Cost: Lets define at each time instance indexed by k, ρ_k as the smallest distance between the relative vector of the quadrotor to the target, $\mathbf{B_k}$ to the nearest obstacle O_n with coordinates $(\mathrm{x}_n, \mathrm{y}_n)$. The occlusion cost function is defined as:

$$J_{occ} = \sum_{k=0}^{N-1} \mathrm{e}^{-\rho_k} \tag{15}$$

The relative vector is defined as:

$$\mathbf{B_K} = \mathbf{P}_k - \mathbf{T}_k \, , k \in \{0, 1, \ldots, N-1\} \tag{16}$$

where P and T are the Quadrotor and target 2-D (x,y) position, respectively. This cost function pushes the bearing vector away from the obstacles, thus minimizing occlusion. The distance is illustrated in Fig. 9.

4) Occlusion Metric: To assess visibility, an occlusion metric is introduced that quantifies how obstacles interfere with the line-of-sight to the target. Note that this metric is used solely for evaluation and is not part of the controller's cost function. This metric allows for an intuitive, interpretable evaluation of visibility as it directly relates to the camera's field of view. Consider the minimum distance between the nearest obstacle and the LOS segment, defined as

$$d = \min_{\lambda \in [0,1]} \|\mathbf{O}_k - (\mathbf{P}_k + \lambda(\mathbf{T}_k - \mathbf{P}_k))\| \, . \tag{17}$$

This distance is mapped to an angular occlusion value $\theta_{\mathrm{occ}} \in [0, \theta_{\mathrm{FOV}}]$, where $\theta_{\mathrm{FOV}} = 40^\circ$ is the camera FOV. A value of $\theta_{\mathrm{occ}} = 0^\circ$ indicates no occlusion, while $\theta_{\mathrm{occ}} = \theta_{\mathrm{FOV}}$ represents a full LOS obstruction. The metric is defined as

$$\theta_{\mathrm{occ}} = \theta_{\mathrm{FOV}} \cdot \max \left(0, \, 1 - \frac{d}{d_{FOV}} \right) \tag{18}$$

where d_{FOV} is the max distance of the camera field of view, which is 4 meters.

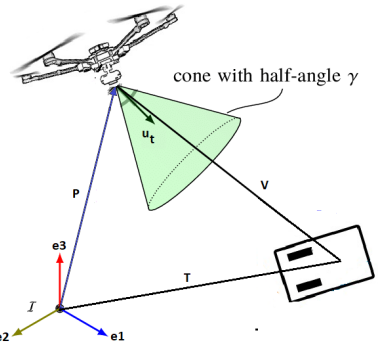


Fig. 10. Illustration of the cone constraint.

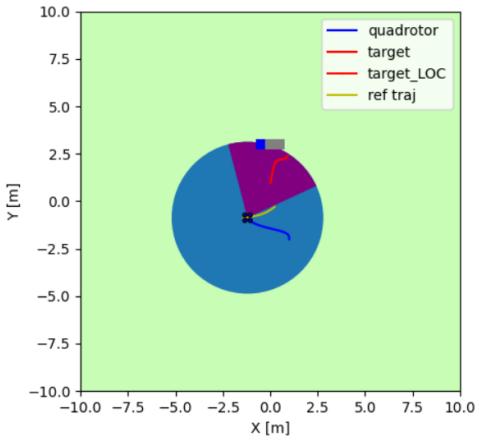


Fig. 11. Illustration of the environment.

E. Field of View Alignment Cost

The camera FOV can be represented as a cone. To ensure that the target remains inside this cone, a cost term is incorporated into the optimization. This term allows the optimization to exploit the coupled angular and translational , in other words it couples perception and actuation. The geometric constraint is illustrated in Fig. 10.

The target is inside the viewing cone if the angle between $\mathbf{V}$ and the camera optical axis $\hat{\mathbf{u_t}}$ must satisfy

$$\angle(\mathbf{V}, \hat{\mathbf{u_t}}) \leq \frac{\theta_{FOV}}{2}.$$

Using the dot product geometric identity, the cone constraint can be written as

$$\mathbf{V}^T \hat{\mathbf{u_t}} \leq \|\mathbf{V}\|_2 \cos(\frac{\theta_{FOV}}{2})$$

Let the body/world alignment that at zero attitude the body axes coincide with the world axes:

$$\text{body } +x = \text{right}, \quad \text{body } +y = \text{forward}, \quad \text{body } +z = \text{up}.$$

Thus, for a camera mounted looking forward, the camera optical axis coincides with the positive body x-axis. Notice that the camera optical axis does not depend on the roll angle. The vehicle attitude is represented by Euler angles: yaw ψ, pitch θ,the camera optical axis in the world frame is defines as

$$\hat{\mathbf{u_t}} = R_z(\psi) R_y(\theta) \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \tag{19}$$

where R_y, and R_z denote the rotation matrices:

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}, \quad R_z(\psi) = \begin{bmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{20}$$

a) Handling Inequality Constraint: Let $\mathrm{s} = \mathbf{V}^T \cdot \hat{\mathbf{u_t}} - \|\mathbf{V}\|_2 \cos(\frac{\theta_{FOV}}{2})$, a natural choice for the penalty function is the max function,

$$\rho_{\text{hard}}(s) = [\max(0,\, s)]^2, \tag{21}$$

which penalizes only when the target is outside the cone. However, this function is non-differentiable at $s = 0$. To preserve differentiability, the penalty functions is approximated by the softplus [11] formulation:

$$\rho_{\text{soft}}(s) = \left[\frac{1}{\beta}\log\left(1 + e^{\beta s}\right)\right]^2, \tag{22}$$

where $\beta = 5$ controls the sharpness of the transition. The final FOV alignment cost is therefore expressed as

$$J_{\text{fov}}(\psi, \theta, \phi, \mathbf{P}, \mathbf{T}) = \lambda_{fov}\left[\frac{1}{\beta}\log\left(1 + e^{\beta s}\right)\right]^2. \tag{23}$$

This cost function is smooth and differentiable. Notice that this cost term only enforces that the target stays inside the FOV, not necessarily centered.

IV. Simulator

The simulated environment is a 2-dimensional 10x10 m square containing the quadrotor, the target and the obstacles. Its coordinates are described in the inertial frame with origin at (0,0) m.

The target's trajectory is constructed by interpolating, with a cubic spline, a set of 2-D way-points with a constant height of 1 meter. The target's velocity is the derivative of the spline.

The obstacles are modeled as circles with radius of 0.5 meters and illustrated by squares. It is assumed the quadrotor is equipped with a 360 degrees camera with visibility of 4 meters for obstacles detection and with a forward-looking camera with field of view (FOV) of 40 degrees and 4 meters of visibility for tracking purposes. The obstacles are assumed to be visualized by the camera if it is inside a circle with radius of 4 meters. Although the environment is 2-dimensional, all the calculations are done in 3-dimensions.

The environment is depicted in Fig. 11. Light blue shade represents the 360 degrees camera; the purple cone represents the forward-looking camera's FOV; Blue are the observed and grey the unobserved obstacles, respectively.

The simulator is based on the dynamics described in section II-A. In the simulator, the control inputs are assumed to reach the desired values instantaneously (neglected dynamics) and directly influence the state space variables. This is a reasonable assumption, because in practice, the quadrotor has a flight

controller onboard that receives those commands and outputs motor commands in much higher frequency than the high-level planner. The parameters for the simulator are defined in table IV

TABLE II
SIMULATION PARAMETERS

Parameter	Symbol	Value	Unit
Camera field of view (angular)	θ_{FOV}	40	deg
Maximum camera FOV distance	d_{FOV}	4	m
Obstacle radius	r_{obs}	0.5	m

V. RESULTS

In Experiments 1 and 2, a single block of obstacles is added. The goal of this experiment is to test the ability of the quadrotor to track the target while avoiding obstacles and minimize occlusion. In experiment 3, a forest like environment is simulated. The goal is to test how the quadrotor behaves in a cluttered environment. Finally, the target trajectory measurements are corrupted by noise, and the quadrotor's response is analyzed for different prediction horizon. In particular, we examine how the prediction horizon affects tracking accuracy, perception performance and robustness to noise. Additionally, the computational performance of the model predictive control framework is assessed as N varies. All experiments were executed with the following gains, until otherwise:

$$Q = \text{diag}\left(\underbrace{100,\ 100,\ 100}_{\text{position } (x,y,z)}, \underbrace{100,\ 100,\ 0}_{\text{angles } (\phi,\theta,\psi)}, \underbrace{200,\ 200,\ 100}_{\text{velocities}}\right)$$

$$Q_f = 5Q$$

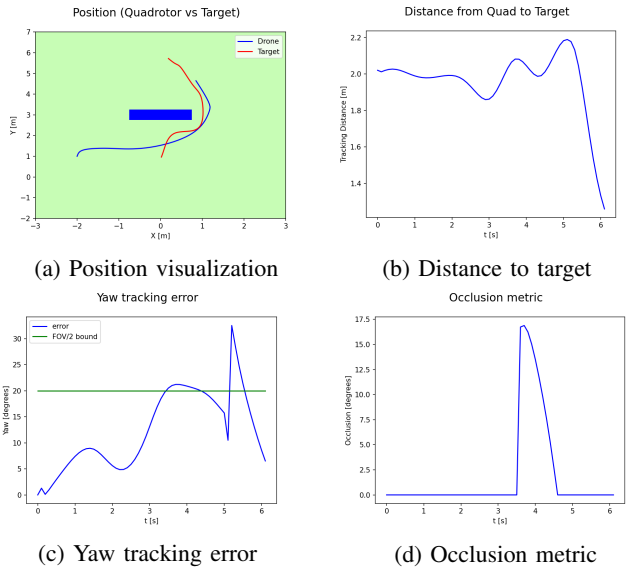
$$R = \text{diag}\left(\underbrace{100}_{\text{collective thrust}}, \underbrace{200,\ 200,\ 100}_{\text{body-rates } (\omega_x,\omega_y,\omega_z)}\right)$$

Parameter	Symbol	Value	Unit
Collision weight	λ_{col}	1000	–
Tracking weight	λ_{tck}	0	–
Occlusion weight	λ_{occ}	0	–
FOV alignment weight	λ_{fov}	100	–

TABLE III
SOFT CONSTRAINTS WEIGHTS

Parameter	Symbol	Value	Unit
Sampling time	Δt	0.1	s
Prediction horizon	N	10	–

TABLE IV
OPTIMIZATION PARAMETERS



(a) Position visualization (b) Distance to target

(c) Yaw tracking error (d) Occlusion metric

Fig. 12. Experiment 1 – Position and velocity results.

A. Experiment 1 - Baseline Obstacle Scenario

The quadrotor starts 2 meters behind and yaw to 0 degrees, with the following initial conditions:

$$P_L(T_0) = [0.0,\ 1.0,\ 1.0]\ \text{m}, \quad V_L(T_0) = [0.2,\ 1.0,\ 0.0]\ \text{m/s},$$
$$P_F(T_0) = [-2.0,\ 1.0,\ 1.0]\ \text{m}, \quad V_F(T_0) = [0.1,\ 0.5,\ 0.0]\ \text{m/s},$$
$$\psi(T_0) = 0^\circ$$

In this experiment, the target moves around the obstacles. This experiment was conducted to test the collision avoidance of the MPC as well as the occlusion minimization. The gains for soft constraints were modified to: λ_{col} = 1000, λ_{occ} = 1000.

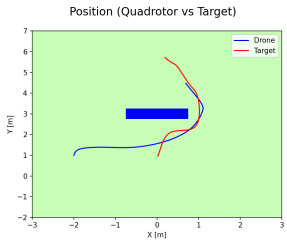
The target moves around the obstacle, causing the drone to avoid the obstacles by executing a longer curve, therefore increasing the tracking distance substantially. In this time period, the quadrotor's yaw deviates from the reference yaw exceeding half of the FOV (Fig. 14(c)) and the FOV is partially occluded (Fig. 14(d)).

B. Experiment 2 - Effect of Varying Occlusion Weights

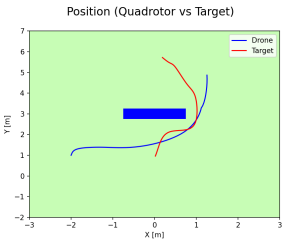
The goal of this section is to analyze the behavior of the quadrotor when the occlusion weight increases. For this experiment, conditions from Experiment 3 holds, varying only the occlusion weight as such: 0, 100, 1000, 10000. The results can be visualized in Figs. 13, 14, 15, 16.

The results with occlusion gains of λ_{occ} = 0, λ_{occ} = 100 and λ_{occ} = 1000 are similar. When λ_{occ} = 10000, the differences are significant, which can be observed in Fig. 13(d). The quadrotor executes a higher right curve, it minimizes occlusion (fig. 14(d)), the yaw error decreases significantly (fig. 15).

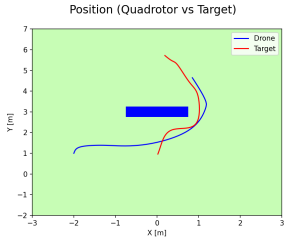
From these results, we can observe that occlusion is influenced only by increasing significantly the occlusion weight. This is true because the quadrotor is trying to track the reference position as well as minimizing the occlusion. Naturally this is a tradeoff. In the next experiment, the tracking cost and xy error weights will be modified and observe how it will affect occlusion.



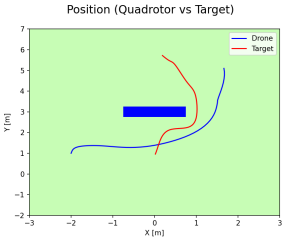
(a) Position comparison $\lambda_{occ} = 0$



(b) Position comparison $\lambda_{occ} = 100$

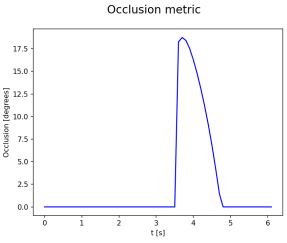


(c) Position comparison $\lambda_{occ} = 1000$

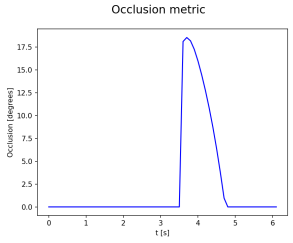


(d) Position comparison $\lambda_{occ} = 10000$

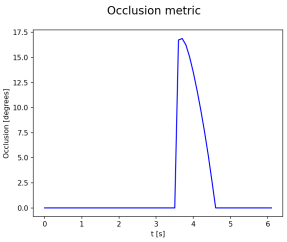
Fig. 13. Experiment 2 – Position comparison with varying occlusion weight.



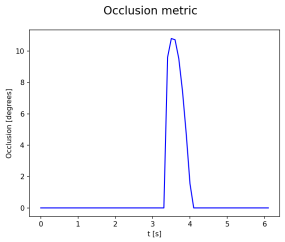
(a) Occlusion metric comparison λ_{occ} = 0



(b) Occlusion metric comparison λ_{occ} = 100

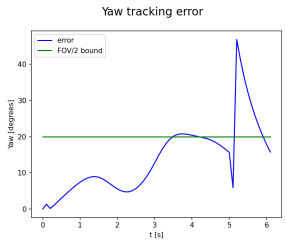


(c) Occlusion metric comparison λ_{occ} = 1000

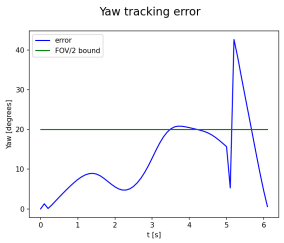


(d) Occlusion metric comparison λ_{occ} = 10000

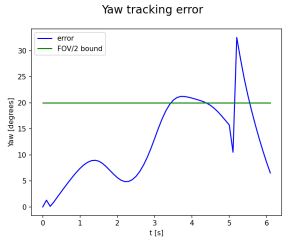
Fig. 14. Experiment 2 – Occlusion metric comparison with varying occlusion weight.



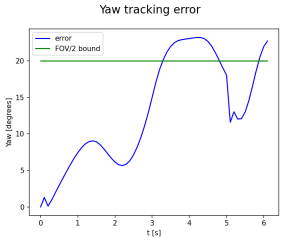
(a) Yaw error comparison λ_{occ} = 0



(b) Yaw error comparison λ_{occ} = 100



(c) Yaw error comparison λ_{occ} = 1000

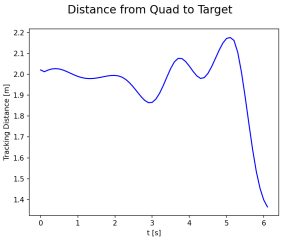


(d) Yaw error comparison λ_{occ} = 10000

Fig. 15. Experiment 2 – Yaw error comparison with varying occlusion weight.



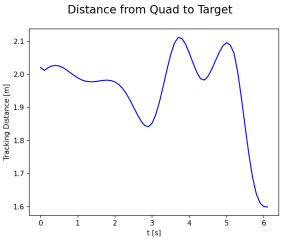
(a) Distance to target comparison λ_{occ} = 0



(b) Distance to target comparison λ_{occ} = 100



(c) Distance to target comparison λ_{occ} = 1000



(d) Distance to target comparison λ_{occ} = 10000

Fig. 16. Experiment 2 – Distance to target comparison with varying occlusion weight.

C. Experiment 3 - Baseline Forest Navigation

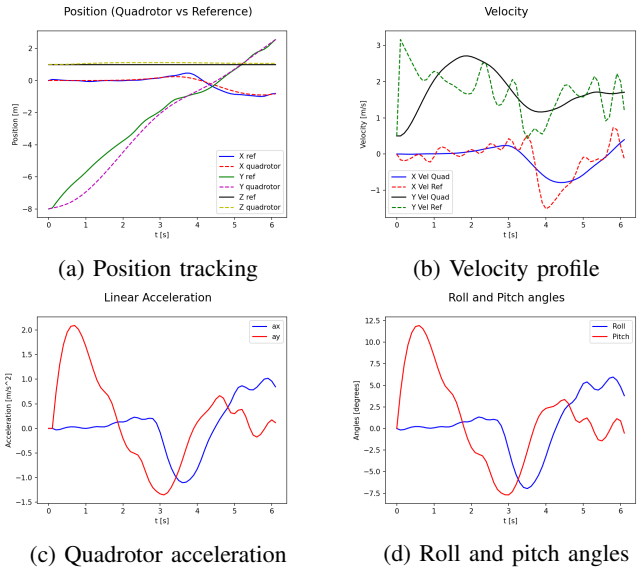
The quadrotor starts 2 meters behind and pointing the camera to 90 degrees with the following initial conditions:

$$P_L(T_0) = [0.0,\ -6.0,\ 1.0]\ \text{m}, \quad V_L(T_0) = [0.2,\ 3.2,\ 0.0]\ \text{m/s},$$
$$P_F(T_0) = [0.0,\ -8.0,\ 1.0]\ \text{m}, \quad V_F(T_0) = [0.0,\ 0.5,\ 0.0]\ \text{m/s},$$
$$\psi_{\text{ref}} = 90^\circ$$

The target's position and velocity profile are shown in Figs. 17(a) and 17(b). In this experiment, the target moves around a simulated forest. This experiment was conducted to test the collision avoidance of the MPC as well as the occlusion minimization in a more dense obstacle environment. The gains for soft constraints were modified to: λ_{col} = 1000, λ_{occ} = 1000 and λ_{fov} = 100.

The results indicate that the MPC is able to maintain effective target tracking and visibility in a cluttered environment. As seen in fig.(18(b)), the tracking distance initially increases during the first second due to obstacle avoidance, before decreasing to around 2 meters.

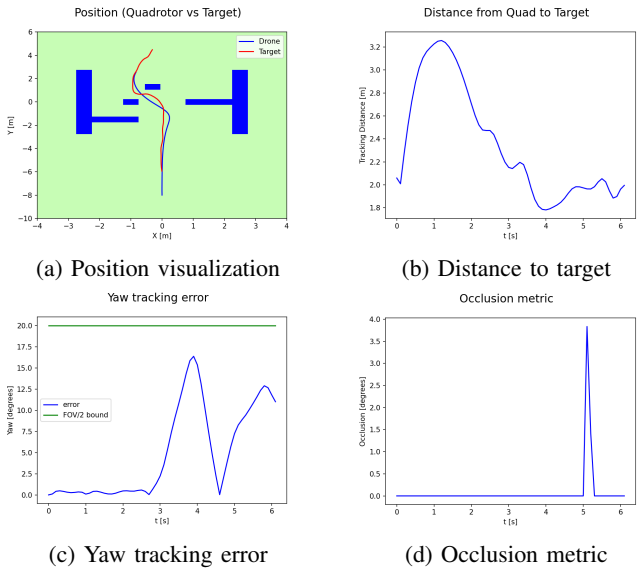
The occlusion metric remains zero for almost the entire experiment fig.(18(d)), showing that the controller consistently selects collision-free viewpoints with a clear line-of-sight. A brief occlusion event at around 5 seconds occurs when the target moves behind a cluster of obstacles, indicating a



(a) Position tracking (b) Velocity profile

(c) Quadrotor acceleration (d) Roll and pitch angles

Fig. 17. Experiment 3 – Translational and rotational dynamics.



(a) Position visualization (b) Distance to target

(c) Yaw tracking error (d) Occlusion metric

Fig. 18. Experiment 3 – Tracking and visibility behavior.

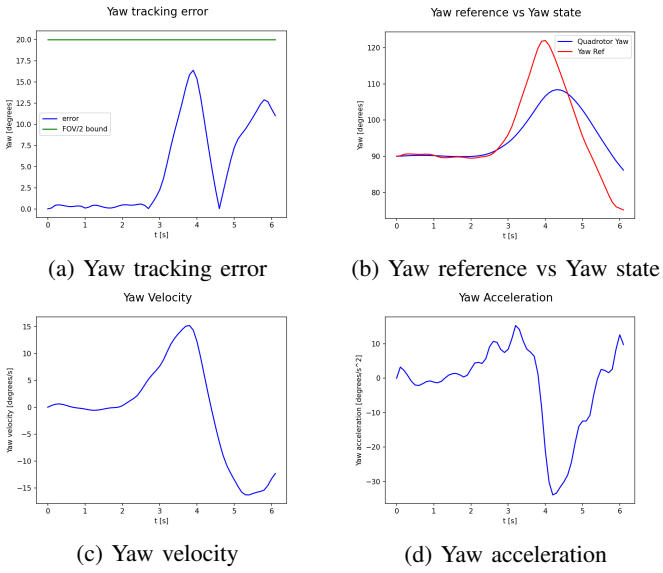
momentary but unavoidable loss of visibility.

Yaw tracking performance fig.(18(c)) shows that the quadrotor closely follows the time-varying yaw reference, with only a short deviation during a sharper target turn.

Overall, the chosen soft-constraint weights ($\lambda_{col} = 1000$, $\lambda_{occ} = 1000$, $\lambda_{fov} = 100$) enable safe and occlusion-aware target tracking with constant distance and yaw reference tracking in a dense obstacle environment.

D. Effects of prediction horizon and noise

The goal of this section is to analyze the behavior of the quadrotor under varying prediction horizon lengths and noisy target waypoints. Gaussian additive white noise was intentionally added to the target waypoints to reflect more realistic tracking conditions. For this experiment, conditions from Experiment 5A holds, varying only the prediction horizon, N to 5, 10, 15, 20.



(a) Yaw tracking error (b) Yaw reference vs Yaw state

(c) Yaw velocity (d) Yaw acceleration

Fig. 19. Experiment 3 – Yaw dynamics.

With a short horizon, the motion target is defined by only a few waypoints and propagated over a short time, making the controller highly sensitive to noise and prone to local trajectory optimizations. This produces oscillatory roll and pitch commands, aggressive body rates, and frequent collisions.

Medium horizons provide greater access to the target motion trajectory, allowing it to be propagated over a longer period, which improves control smoothness, target tracking, and occlusion avoidance. Very long horizons further enhance smoothness and reduce sensitivity to waypoint noise, but sudden target maneuvers may be tracked more slowly. Additionally, the extended prediction can lead to larger tracking errors than moderate horizons because the controller prioritizes smooth, long-term trajectories and under-reacts to short-term target variations, while moderate horizons better balance responsiveness and control smoothness.

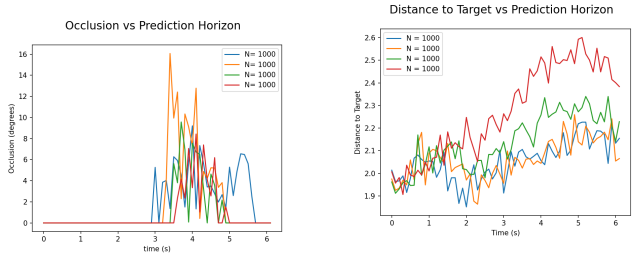
Horizon length also affects distance to target and the ability to maintain the target within the field of view. Medium horizons also achieve the most effective occlusion avoidance, maintaining clear line-of-sight to the target while balancing responsiveness. With a longer horizon, trajectories are planned over a long future period, emphasizing smoothness and noise rejection. As a result, the quadrotor may deviate from the target, increasing distance and reducing FOV coverage.

E. Computational Performance

In this section, the computational performance of model predictive control was evaluated for different prediction horizon lengths N, using the setup from Experiment 3.

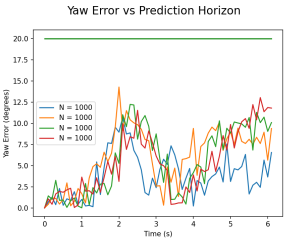
The solver was implemented in Python using CasADi as the automatic differentiation and optimization interface, and IPOPT was used as the nonlinear programming solver. All computations were performed on a laptop equipped with an Intel Core i7 processor running at 2.60 GHz.

Table V summarizes the minimum, average, and maximum solver execution times per cycle. As expected, the computational burden increases with N, since a longer prediction

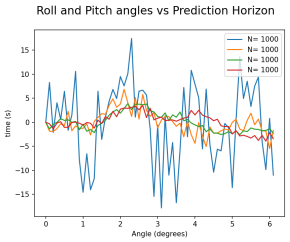


(a) Occlusion comparison

(b) Distance to target comparison



(c) Field of View Error comparison



(d) Roll and Pitch angles comparison

Fig. 20. Experiment 3 – Behavior comparison with varying prediction horizon.

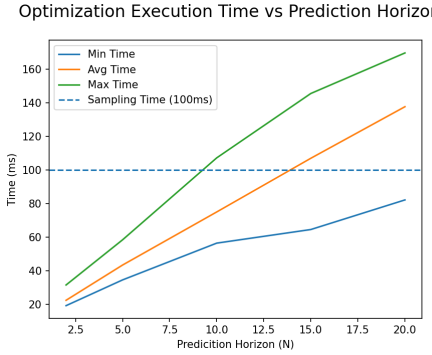


Fig. 21. Computation time vs Prediction Horizon

horizon results in a larger nonlinear optimization problem, as shown in Fig. 21.

TABLE V
SOLVER EXECUTION TIMES FOR DIFFERENT PREDICTION HORIZON LENGTHS

N	**Min** (ms)	**Avg** (ms)	**Max** (ms)
2	19.26	22.46	31.57
5	34.59	43.44	58.46
10	56.49	75.05	107.27
15	64.60	107.04	145.62
20	82.23	137.75	169.76

A sampling period of $T_s = 0.1$ s (100 ms) was imposed as the real-time computation constraint. For horizons up to $N = 10$, the average solver time remains below this limit. However, for $N \geq 15$, both the average and maximum execution times exceed the sampling budget, indicating that real-time feasibility is no longer guaranteed. The results indicate that the average execution time of prediction horizons up to approximately $N = 10$ are below the sampling time. Larger horizons would require either increased sampling time, reduced solver tolerances, warm-starting strategies, or simplifications in the dynamics and constraints. It should be noted however that these results are obtained on a general-purpose laptop CPU and therefore cannot be directly extrapolated to embedded hardware platforms. Differences in processor architecture, memory and real-time operating conditions can lead to significantly different execution times.

VI. Conclusions

In this thesis we propose trajectory generation method allows a quadrotor to follow a target while avoiding obstacles. The problem is formulated as a receding horizon optimal control program with non-linear constraints. We have demonstrated in simulation that the algorithm can satisfy dynamic constraints, continuously minimize occlusions from obstacles in the environment and avoid collisions with these. Analysis of prediction-horizon lengths revealed a key tradeoff between responsiveness and smoothness: short horizons react quickly but amplify noise and aggressive behavior, whereas long horizons improve stability and reduce noise sensitivity at the cost of slower adaptation. A moderate horizon provided the best compromise, enabling reliable tracking, smooth motion, and robust visibility maintenance. Finally, the computational performance of model predictive control was evaluated for different prediction horizon lengths. For N 15, both the average and maximum execution times ex- ceed the sampling budget, indicating that real-time feasibility is no longer guaranteed.

References

[1] D. Falanga, P. Foehn, P. Lu, and D. Scaramuzza, "Pampc: Perception-aware model predictive control for quadrotors," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 1–8.

[2] R. Mahony, V. Kumar, and P. Corke, "Multirotor aerial vehicles: Modeling, estimation, and control of quadrotor," *IEEE Robotics Automation Magazine*, vol. 19, no. 3, pp. 20–32, 2012.

[3] C. Nissoux, T. Simeon, and J.-P. Laumond, "Visibility based probabilistic roadmaps," in *Proceedings 1999 IEEE/RSJ International Conference on Intelligent Robots and Systems. Human and Environment Friendly Robots with High Intelligence and Emotional Quotients (Cat. No.99CH36289)*, vol. 3, 1999, pp. 1316–1321 vol.3.

[4] S. Lavalle and J. Kuffner, "Rapidly-exploring random trees: Progress and prospects," *Algorithmic and computational robotics: New directions*, 01 2000.

[5] M. Likhachev, G. J. Gordon, and S. Thrun, "Ara*: Anytime a* with provable bounds on sub-optimality," in *NIPS*, 2003.

[6] J. Chen, T. Liu, and S. Shen, "Tracking a moving target in cluttered environments using a quadrotor," in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, pp. 446–453.

[7] L. Xi, Z. Peng, L. Jiao, and B. Chen, "Smooth quadrotor trajectory generation for tracking a moving target in cluttered environments," *Science China Information Sciences*, vol. 64, 07 2021.

[8] P. Pereira, R. Cunha, D. Cabecinhas, C. Silvestre, and P. Oliveira, "A 3-d trailer approach to leader-following formation control," *IEEE Transactions on Control Systems Technology*, vol. 28, no. 6, pp. 2292–2308, 2020.

[9] P. O. Pereira, R. Cunha, D. Cabecinhas, C. Silvestre, and P. Oliveira, "Leader following trajectory planning: A trailer-like approach," *Automatica*, vol. 75, pp. 77–87, 2017. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0005109816303363

[10] M. Schwenzer, M. Ay, T. Bergs, and D. Abel, "Review on model predictive control: an engineering perspective," *The International Journal of Advanced Manufacturing Technology*, vol. 117, pp. 1–23, 11 2021.

[11] I. Askari and S. Zeng, "Nonlinear model predictive control based on constraint-aware particle filtering/smoothing," 05 2022.